

PRACTICAL WEEK 03

1. Develop a C program using `fork()` that creates a parent and child process. Displays the Process ID (PID), Parent Process ID (PPID), and process states at different stages at different stages of execution.

```
root@SRIVALLI:~# nano fork_process.c
root@SRIVALLI:~# gcc fork_process.c -o fork_process
root@SRIVALLI:~# ./fork_process

Parent Process

Parent PID: 522
Child Process
Child PID: 523
Parent is waiting for child...
Child PID: 523
Parent PID: 522
Child is running...
Child process completed.
Child process has completed.
Parent process completed.
```

Report:

In this task, I used the `fork()` system call in C to create a parent process and a child process. The program displayed the PID of the parent and child processes and also showed the Parent Process ID (PPID). The `wait()` function was used so that the parent process waits for the child process to finish. This helped me understand how processes are created and how parent and child processes work in Linux.

2.Design an experiment to observe process state transitions(Ready,Running,Waiting,Terminated)using Linux monitoring tools such as ps,top,and/proc.Document the observations.

```

root@SRIVALLI: ~
top - 10:17:24 up 53 min,  1 user,  load average: 0.00, 0.01, 0.00
Tasks:  21 total,   1 running, 20 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.
MiB Mem :  7784.6 total,  7193.9 free,   507.7 used,   233.5 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  7276.9 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+
109 root        20   0  25152  6596  5184 S   0.3   0.1   0:01.56
   1 root        20   0  21796 13252  9764 S   0.0   0.2   0:02.20
   2 root        20   0   3180  2200  2072 S   0.0   0.0   0:00.03
   6 root        20   0   3196  2128  2008 S   0.0   0.0   0:00.00
  58 root        19  -1 34064 12616 11536 S   0.0   0.2   0:00.53
126 systemd+   20   0  21344 13192 11064 S   0.0   0.2   0:00.47
127 systemd+   20   0  91036  7960  6988 S   0.0   0.1   0:00.39
193 root        20   0   4244  2684  2432 S   0.0   0.0   0:00.02
194 message+   20   0   9544  5388  4692 S   0.0   0.1   0:00.17
205 root        20   0  17980  8856  7816 S   0.0   0.1   0:00.30
231 syslog     20   0 222516  5932  4612 S   0.0   0.1   0:00.27
235 root        20   0   3124  1976  1836 S   0.0   0.0   0:00.02
241 root        20   0 107036 23252 13796 S   0.0   0.3   0:00.46
335 root        20   0   3184  1104   980 S   0.0   0.0   0:00.00
338 root        20   0   3200  1244  1108 S   0.0   0.0   0:00.07
342 root        20   0   6592  5872  3656 S   0.0   0.1   0:00.04
344 root        20   0   6668  4576  3812 S   0.0   0.1   0:00.02
406 root        20   0  20324 11512  9388 S   0.0   0.1   0:00.14
407 root        20   0  21172  3496  1784 S   0.0   0.0   0:00.00
431 root        20   0   6080  5284  3648 S   0.0   0.1   0:00.02
758 root        20   0   9300  5700  3512 R   0.0   0.1   0:00.66

```

Report: In this task, I used the fork() system call in C to create a parent process and a child process. The program displayed the PID of the parent and child processes and also showed the Parent Process ID (PPID). The wait() function was used so that the parent process waits for the child process to finish. This helped me understand how processes are created and how parent and child processes work in Linux.